

Project on

Physical Design Implementation of

Serial Peripheral Interface

Using Q flow

Name:- Yarra Bhargav

Course:- Maven Silicon PD Internship

Batch:- B2B PDI-01

Table of Contents

1. Introduction
2. Serial Peripheral Interface
3. Invoke the tool
4. Synthesis
5. Placement
6. Static Timing Analysis
7. Routing
8. DRC (Design Rule Check)
9. LVS
10. GDS
11. Final Output

Introduction

Qflow –An Open Source Digital Synthesis flow

A digital synthesis flow is a set of tools and methods used to turn a circuit design written in a high-level behavioral language like **verilog** or **VHDL** into a physical circuit, which can either be configuration code for an FPGA target like a Xilinx or Altera chip, or a layout in a specific fabrication process technology, that would become part of a fabricated circuit chip. Several digital synthesis flows targeting FPGAs are available, usually from the FPGA (Field Programmable Gate Arrays) manufacturers, and while they are typically not open source, they are generally distributed for free (presumably on the optimistic assumption that more people will be buying more FPGA hardware).

Qflow is a complete tool chain for synthesizing digital circuits starting from verilog source and ending in physical layout for a specific target fabrication process. Simple enough to be used by students in beginner-level digital logic courses, yet powerful enough to be included into large EDA systems like Cadence or Synopsys, commercial electronics designers and researchers in cutting-edge environments have relied on Qflow for several years to maintain quality and low hardware costs. Qflow is an open source tool that has been used in digital circuit and system design classes at universities and colleges around the world, and some small businesses and individuals also use it as part of in-house silicon design. The algorithms for digital design flow are still locked up in closed source software, and development is hard outside the domain of the big EDA tools vendors. Qflow is an excellent example of how, when the best components that are available are open-source, and that can be maintained by the user within the flow, then better alternatives to any component exist.

Qflow consists of some list of tools like:

- **Yosys** – RTL Synthesis
- **blif2struct** – High Format Net Synthesis
- **graywolf** – Placement
- **magic** – VLSI Layout Tool
- **netgen** – LVS Layout vs Schematic
- **OpenSTA** – Static Timing Analysis tool

Advantages of Qflow :-

1.Improved Efficiency: Automates workflow processes, reducing manual errors and streamlining operations, which leads to faster service delivery.

2.Enhanced Customer Experience: Provides a seamless journey for customers from appointment scheduling to service completion, improving satisfaction and engagement.

3.Scalability: Capable of handling high volumes of appointments and queues, making it suitable for both small businesses and large enterprises.

4.Robust Analytics: Offers comprehensive analytics and insights, enabling organizations to make data-driven decisions and optimize their operations.

5.Flexible Deployment Options: Supports various deployment models (cloud, on-premise, or hybrid), allowing organizations to choose the best fit for their needs.

Serial Peripheral Interface

Serial Peripheral Interface, or **SPI**, is a very common communication protocol used for two-way communication between two devices. A standard SPI bus consists of 4 signals:

- **MOSI** (Master Out Slave In)
- **MISO** (Master In Slave Out)
- **SCLK** (Serial Clock)
- **SS** (Slave Select)

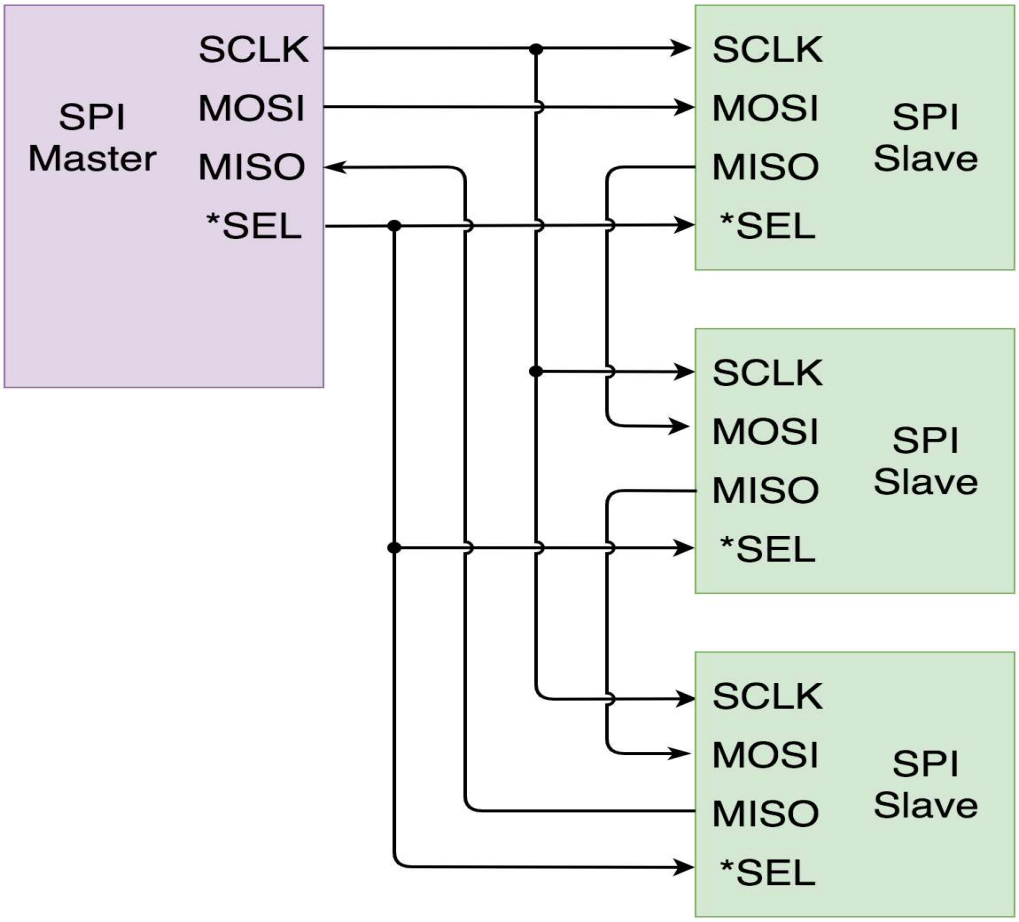
Unlike an asynchronous serial interface, SPI is not convenient. An SPI bus has one master and one or more slaves. The master can talk to only one slave on the bus, but each slave can only talk to the master. Each slave on the bus must have its own unique slave select signal. The master uses the slave select signals to select which slave it will be talking to. Since SPI also includes a clock signal, both devices do not need to agree on a data transfer baud rate. The only requirement is that the clock is lower than the maximum frequency for all devices connected.

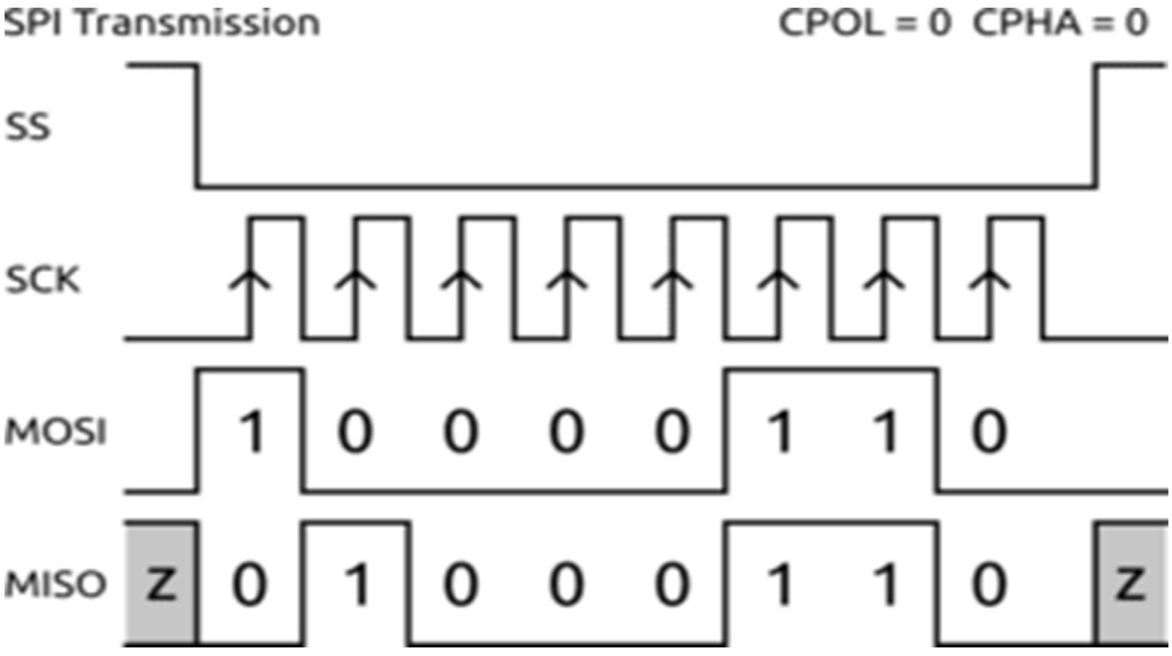
There are 4 different SPI bus standards that all have to do with the **SCK signal**.
The 4 modes are broken down into two parameters, **CPOL** and **CPHA**.

- **CPOL** stands for Clock Polarity and designates the default value (high/low) of the SCK signal when the line is idle.
- **CPHA** stands for Clock Phase and determines which edge of the clock signals data capture (rising/falling).

The slave or device must specify these parameters so you can adjust accordingly.
The most common settings are **CPOL=0 (idle low)** and **CPHA=0 (sample rising edge)**.

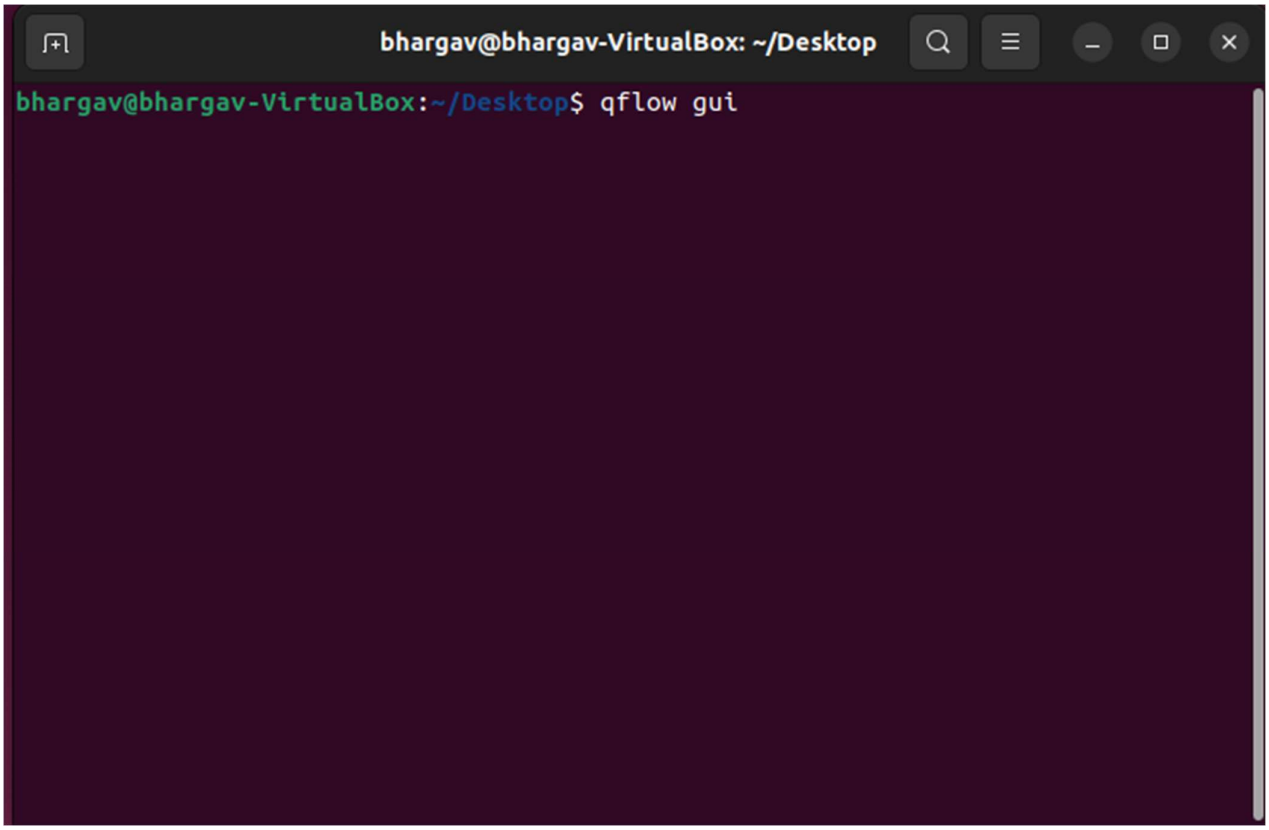
A diagram will help you understand the following in a better way





Invoke the Tool

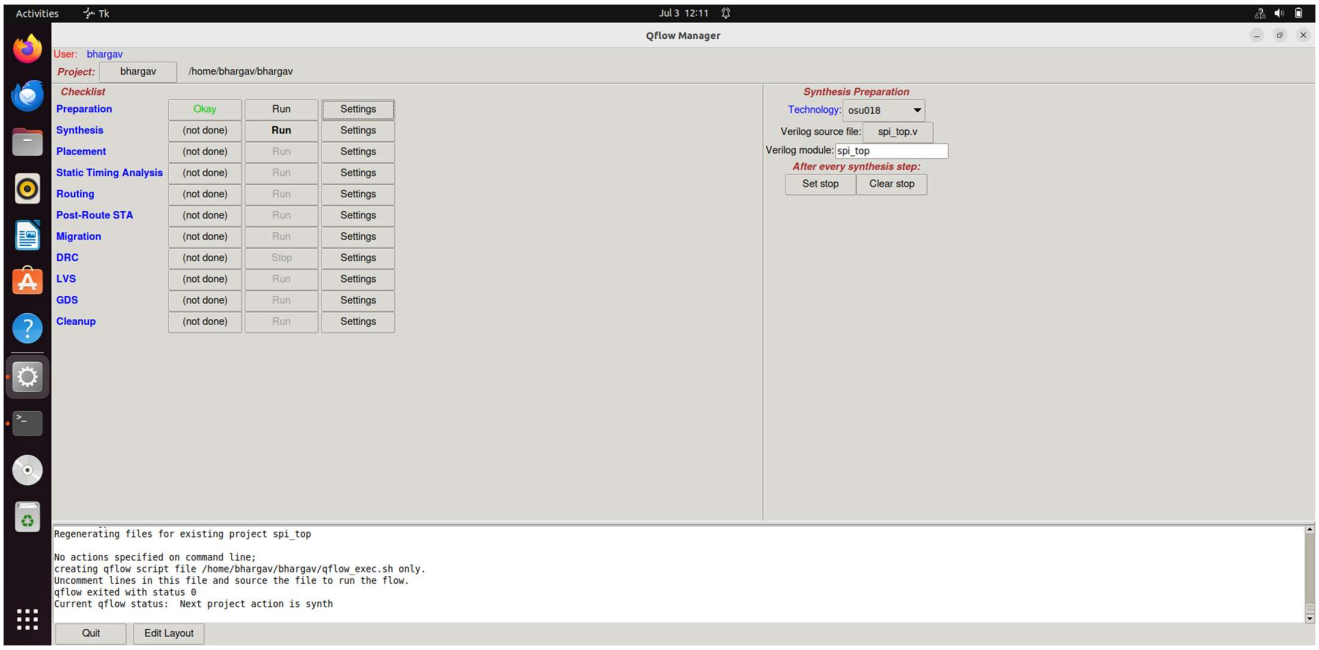
To invoke the tool we require terminal box



After opening the terminal box we have to type “qflow gui” and press enter. It invokes the Qflow tool interface.

Qflow Gui :

Qflow GUI is a graphical user interface designed to work with the Qflow EDA (Electronic Design Automation) tools suite. It provides a user-friendly layer for interacting with Qflow’s tools and processes, aiming to simplify the workflow of designing digital circuits and systems. The GUI is an extension of the powerful features of the Qflow toolchain more accessible to users who prefer visual interactions over command-line operations.



Using the Qflow GUI Tool

The **Qflow GUI** is a graphical interface that helps us run all the important steps involved in digital chip design, especially when working on physical design. Instead of typing commands one by one in the terminal, the Qflow GUI gives us buttons and menus to work with, which is easier for many users.

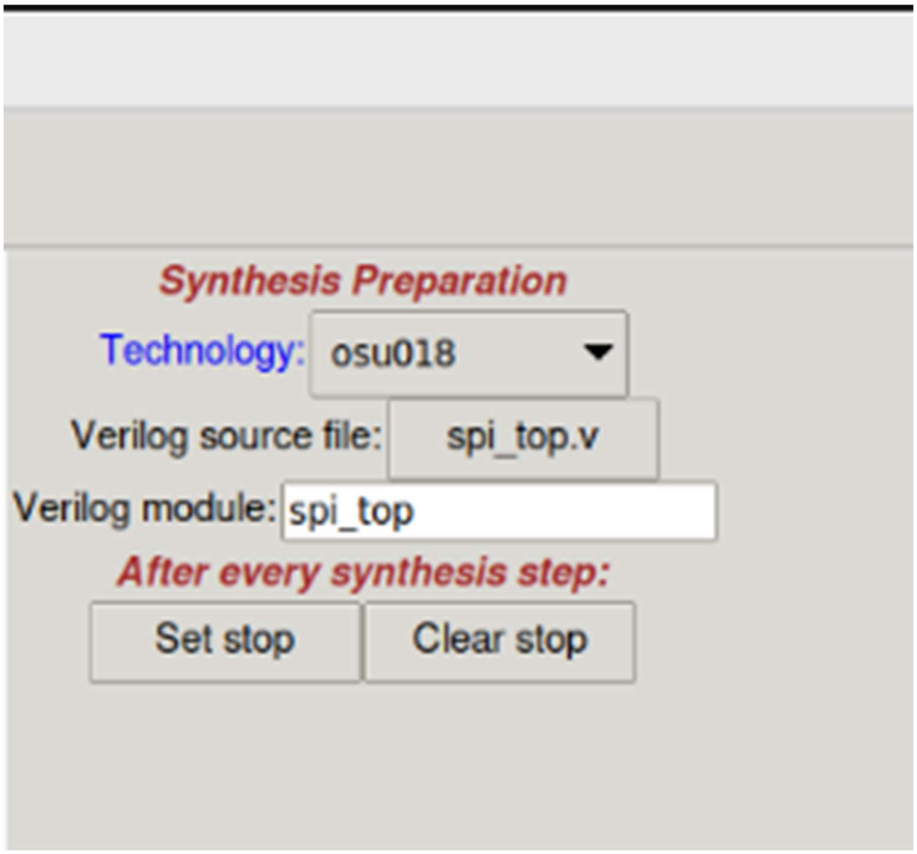
Once we open the Qflow GUI, we'll see a list of steps (or tools) that need to be run in a specific order — like synthesis, placement, routing, etc. Each tool must be run one after the other. We can do this by clicking on the **"Run"** button next to each step.

If the tool runs successfully, it will show a message saying **"OKAY"** in the status column.
If something goes wrong, it will show **"FAIL"** instead. This means we need to check what went wrong, fix the issue (usually a syntax error or a missing file), and try running it again.

Before Running Tools

Before we start running any tool in Qflow, there are a few setup steps we must complete:

1. **Select the Technology File**
   - This file contains all the physical and electrical rules of the chip (like dimensions, layers, spacing, etc.).
   - You'll see names like osu018 or osu035 — choose the one that matches your design process.
2. **Choose the Top Module of Your Verilog File**
   - Your design will be written in Verilog (.v file). You need to tell Qflow which is the **top-level module** (the main circuit you're working with).
   - Make sure your Verilog code is error-free. If there are syntax mistakes, fix them first.



After selecting the technology and Verilog file, you're ready to begin.

- Start by clicking **"Run Preparation"**.
- Once you run this step, Qflow will automatically create several folders inside your project directory. These folders include:
  - layout/ – for layout files
  - source/ – your input Verilog files
  - synthesis/ – synthesis reports
  - log/ – log files from each step
  - tmp/ or tech/ – temporary files or tech-specific data

These folders help organize your project and make it easy to debug if something fails.

Synthesis

Synthesis is a key part of the digital design process. It's the step where high-level code (written in languages like Verilog or VHDL) is turned into a gate-level version that can be built into actual hardware, such as FPGAs or ASICs. In Qflow, this step is done using a tool called **Yosys**, and the **Qflow GUI** makes it easier by giving a visual interface to run and manage it.

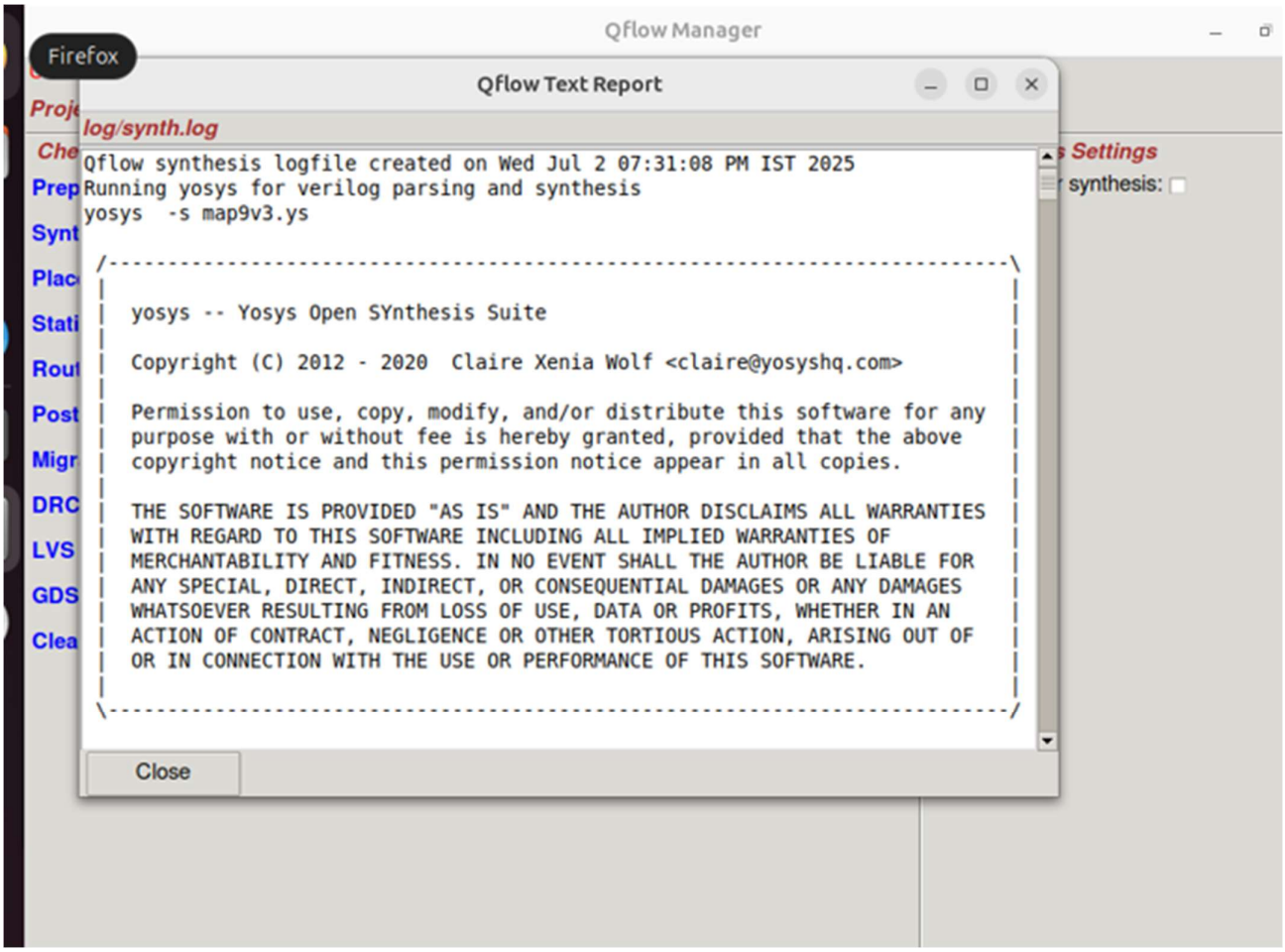
But before running synthesis, make sure your files are saved properly and the project path is correct. Otherwise, errors may occur. You also need to save all your Verilog files in one place and give the folder path in Qflow. After that, click on the synthesis tool. Then wait — if everything goes fine, you'll see **OKAY**.

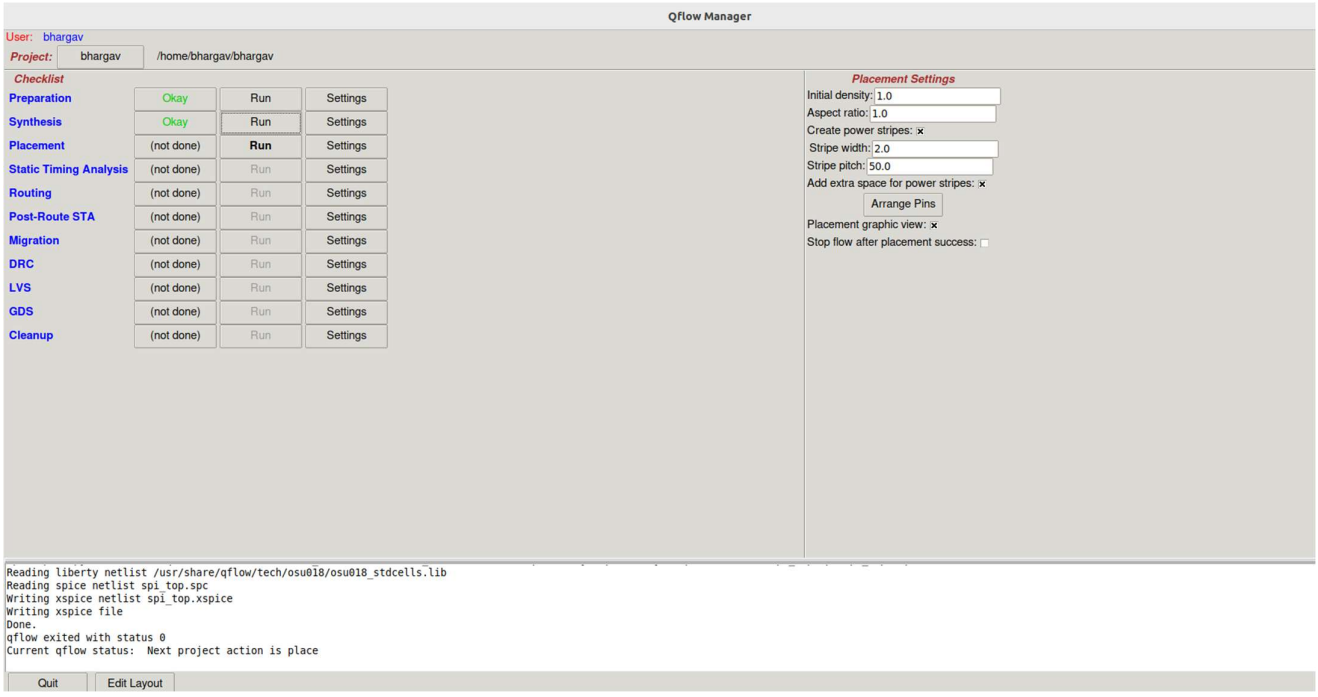
Yosys:-

Yosys is the main synthesis tool used in the Qflow design setup. It helps convert high-level Verilog (HDL) code into a gate-level design. This means it takes your written logic and turns it into actual circuit structure.

Yosys reads the input design, processes it, and creates an output that shows how the hardware will look (called a gate-level netlist). It supports different file formats like BLIF, which is needed for the next steps like placement, routing, and simulation.

Yosys also helps catch syntax errors and supports features like hierarchy checking and optimizations. It provides extra files (like logs and reports) that can be useful to understand what's going on, debug errors, and improve your design.


Qflow Manager
Firefox
Qflow Text Report
log/synth.log
Qflow synthesis logfile created on Wed Jul 2 07:31:08 PM IST 2025
Running yosys for verilog parsing and synthesis
yosys -s map9v3.ys
yosys -- Yosys Open SYnthesis Suite
Copyright (C) 2012 - 2020 Claire Xenia Wolf <claire@yosyshq.com>
Permission to use, copy, modify, and/or distribute this software for any purpose with or without fee is hereby granted, provided that the above copyright notice and this permission notice appear in all copies.
THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY SPECIAL, DIRECT, INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.
Close
Settings
synthesis:


Qflow Manager
User: bhargav
Project: bhargav
/home/bhargav/bhargav
Checklist
Preparation Okay Run Settings
Synthesis Okay Run Settings
Placement (not done) Run Settings
Static Timing Analysis (not done) Run Settings
Routing (not done) Run Settings
Post-Route STA (not done) Run Settings
Migration (not done) Run Settings
DRC (not done) Run Settings
LVS (not done) Run Settings
GDS (not done) Run Settings
Cleanup (not done) Run Settings
Placement Settings
Initial density: 1.0
Aspect ratio: 1.0
Create power stripes:
Stripe width: 2.0
Stripe pitch: 50.0
Add extra space for power stripes:
Arrange Pins
Placement graphic view:
Stop flow after placement success:
Reading liberty netlist /usr/share/qflow/tech/osu018/osu018_stdcells.lib
Reading spice netlist spi_top.spc
Writing xspice netlist spi_top.xspice
Writing xspice file
Done.
qflow exited with status 0
Current qflow status: Next project action is place
Quit
Edit Layout

Placement

Placement is the process of arranging all the standard cells present in the **netlist** into the **core area** of the chip. This is typically handled by EDA tools, which optimize the placement for various design goals.

Types of Placement Techniques

1. **Timing-Driven Placement**
    - Focuses on minimizing timing delays and meeting timing constraints.
    - Prioritizes critical path optimization.
2. **Congestion-Driven Placement**
    - Aims to reduce routing congestion and avoid routing hotspots.
    - Prioritizes wire length and routing resources.
3. **Power-Driven Placement**
    - Targets low power consumption.
    - Places cells to reduce dynamic and leakage power.

Goals of Placement

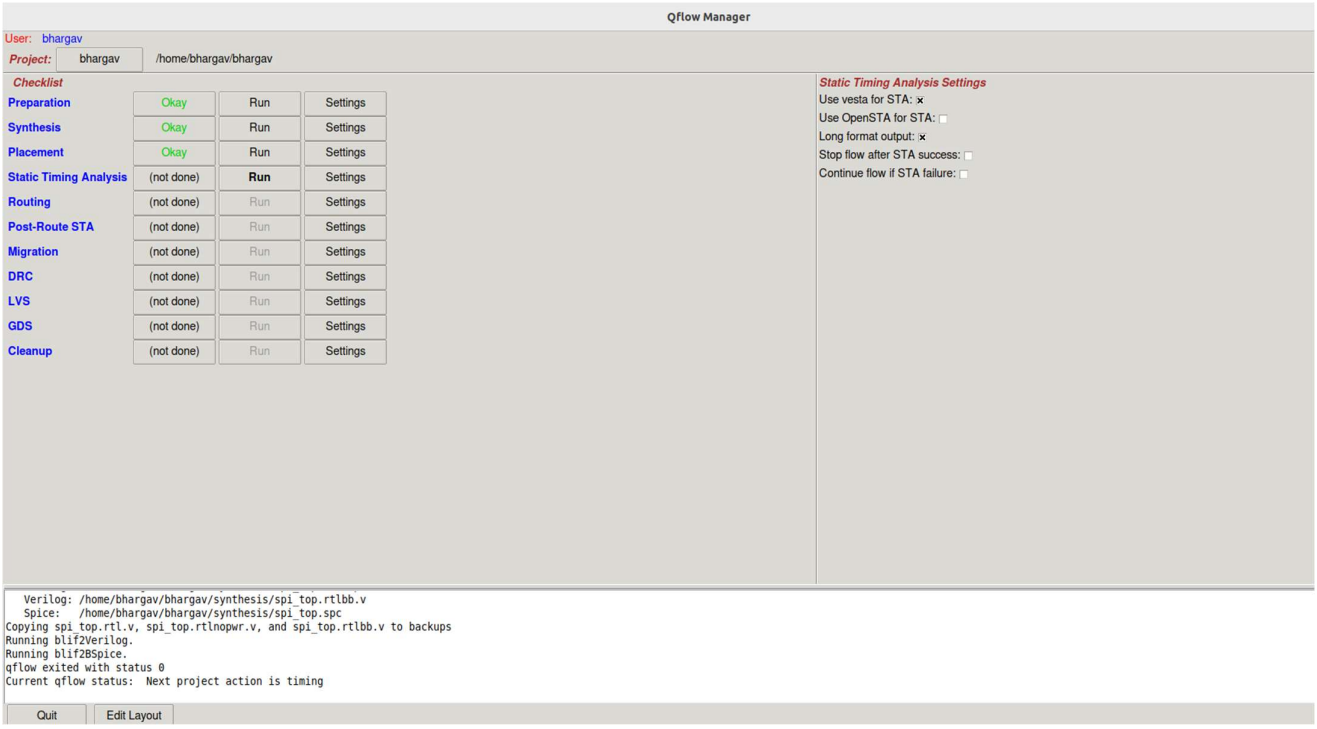
- **Timing optimization** – ensure setup/hold timing is met.
- **Area optimization** – efficient use of chip real estate.
- **Power optimization** – reduce switching and leakage power.
- **Minimize congestion** – avoid routing difficulties.
- **Reduce cell and pin density** – avoid overlap or overuse of routing channels.
- **Avoid timing DRCs** – no design rule violations post placement.

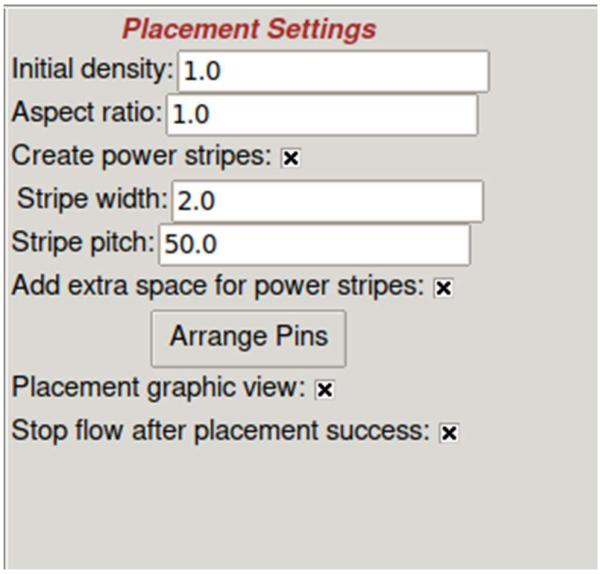
Inputs to Placement

- **Netlist** – the logical connectivity of the design.
- **Physical Libraries (LEF files)** – provide standard cell dimensions and pin positions.
- **Floorplan DEF file** – defines core area, rows, and placement constraints.

Placement Steps

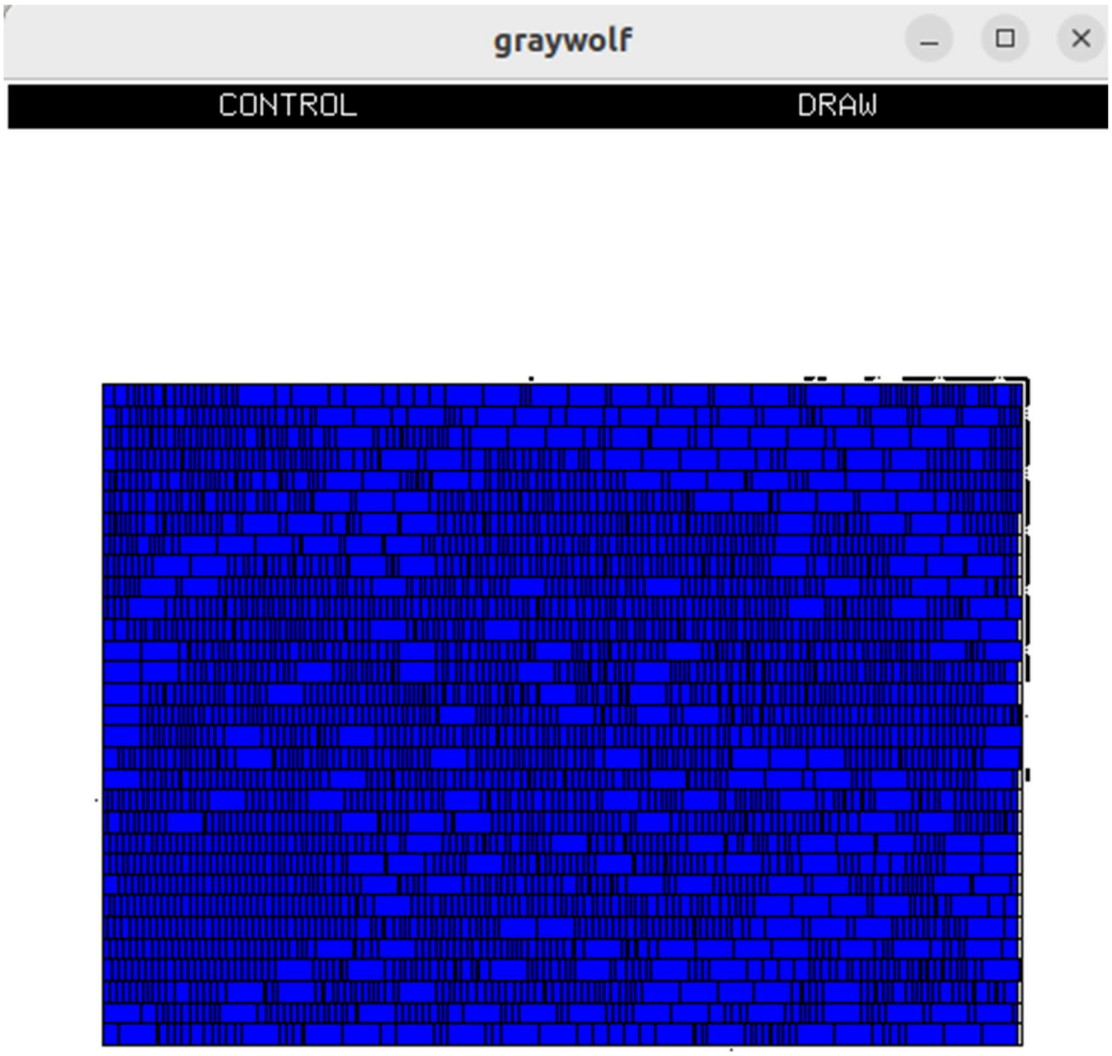
1. **Pre-Placement** – Prepare the floorplan, insert power rails, and reserve space.
2. **Placement** – The actual cell placement using algorithms.
3. **Post-Placement** – Perform optimization, legalize the placement, and check for DRCs.





Graywolf

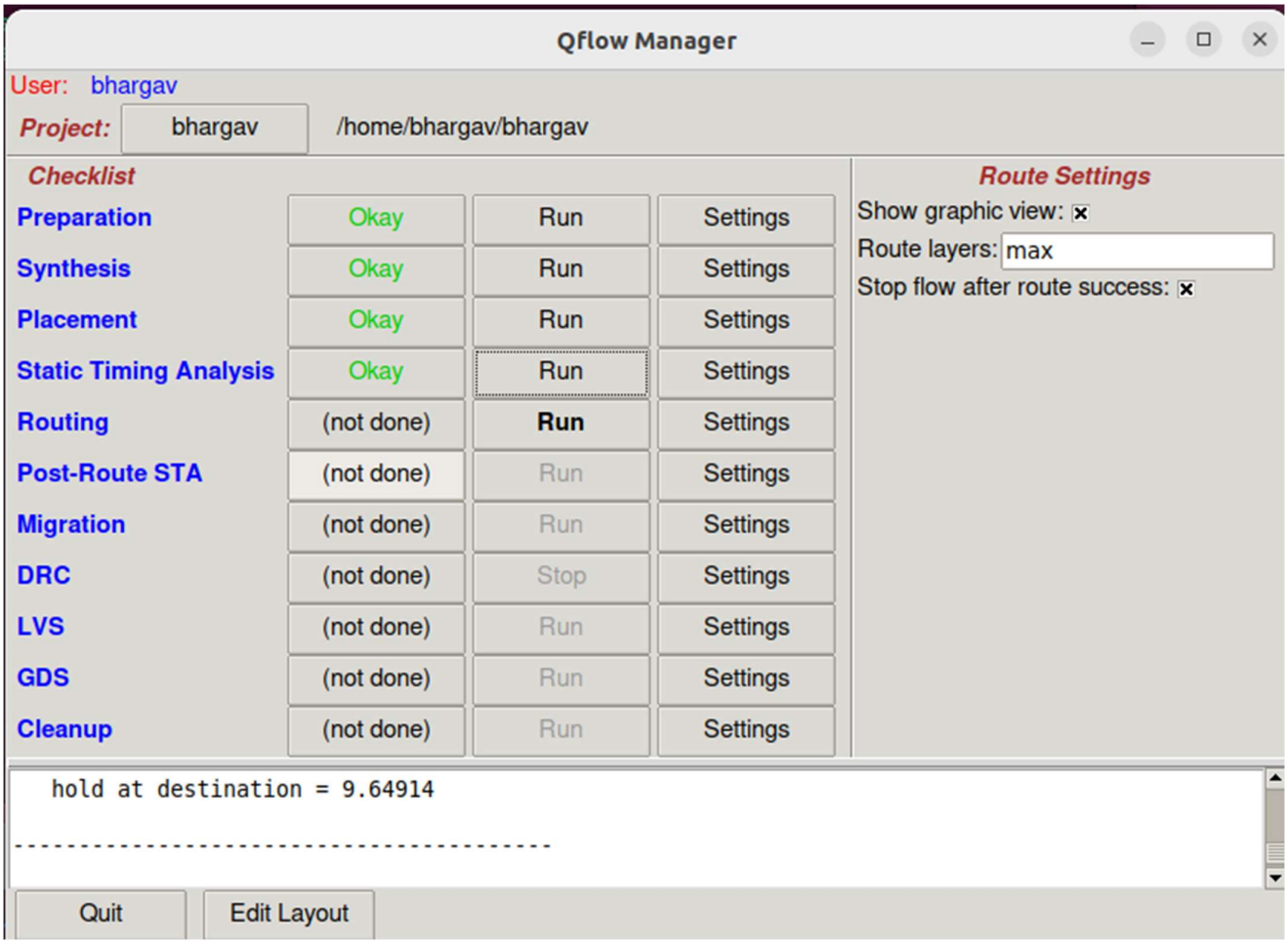
In the Qflow toolchain, **Graywolf** is the placement tool responsible for determining the optimal physical locations of logic cells during the **ASIC** or **FPGA** design process. It plays a critical role in the physical design phase by arranging cells strategically within the chip's core area to ensure high performance and efficient resource usage. The primary goal of Graywolf is to minimize signal delays and reduce routing congestion by placing cells in a way that shortens the distances which are inter connected . It uses sophisticated algorithms to balance cell distribution, control cell density, and maintain uniform pin spacing, all while adhering to design constraints such as timing and power. By achieving a balanced and optimized placement, Graywolf helps ensure that the design not only fits within the available silicon area but also operates with minimal timing violations and maximum routing efficiency. This makes it an essential component of the Qflow suite, directly influencing the overall success and performance of the final chip layout.


graywolf
CONTROL
DRAW

Static Timing Analysis (STA)

Static Timing Analysis (STA) is a widely used method in digital circuit design to **verify the timing performance of a chip**. It checks whether the circuit will operate correctly at the required clock speed, without using test inputs or simulation. Instead, STA analyzes all possible timing paths in the design by calculating delays through gates and interconnections. It then compares these delays with the timing requirements, such as clock period, setup time, and hold time. STA helps **detect critical timing issues** like setup violations (when signals arrive too late) and hold violations (when signals arrive too early). It focuses on the longest and shortest data paths between flip-flops to ensure reliable signal transfer. This process ensures that the design will function properly under worst-case conditions, such as voltage or temperature variations. STA is faster and more efficient than traditional simulation methods, and it is used in almost every step of the VLSI design flow. Tools like Synopsys **PrimeTime** or **OpenTimer** are commonly used to perform static timing analysis in real-world projects.

OpenSTA is an open-source Static Timing Analysis tool that is integrated into the **OpenROAD** project for design automation and optimization. By performing STA on a synthesized netlist and checking against timing constraints, OpenSTA helps ensure timing correctness in the digital design process. By enabling detailed timing analysis during the design cycle, engineers can identify critical paths, optimize performance, and reduce the risk of late-stage design changes. Tools like OpenSTA are important in bringing robust STA capabilities to the design community, enabling both students and professionals to explore STA and the vital role it represents in speed and reliability.

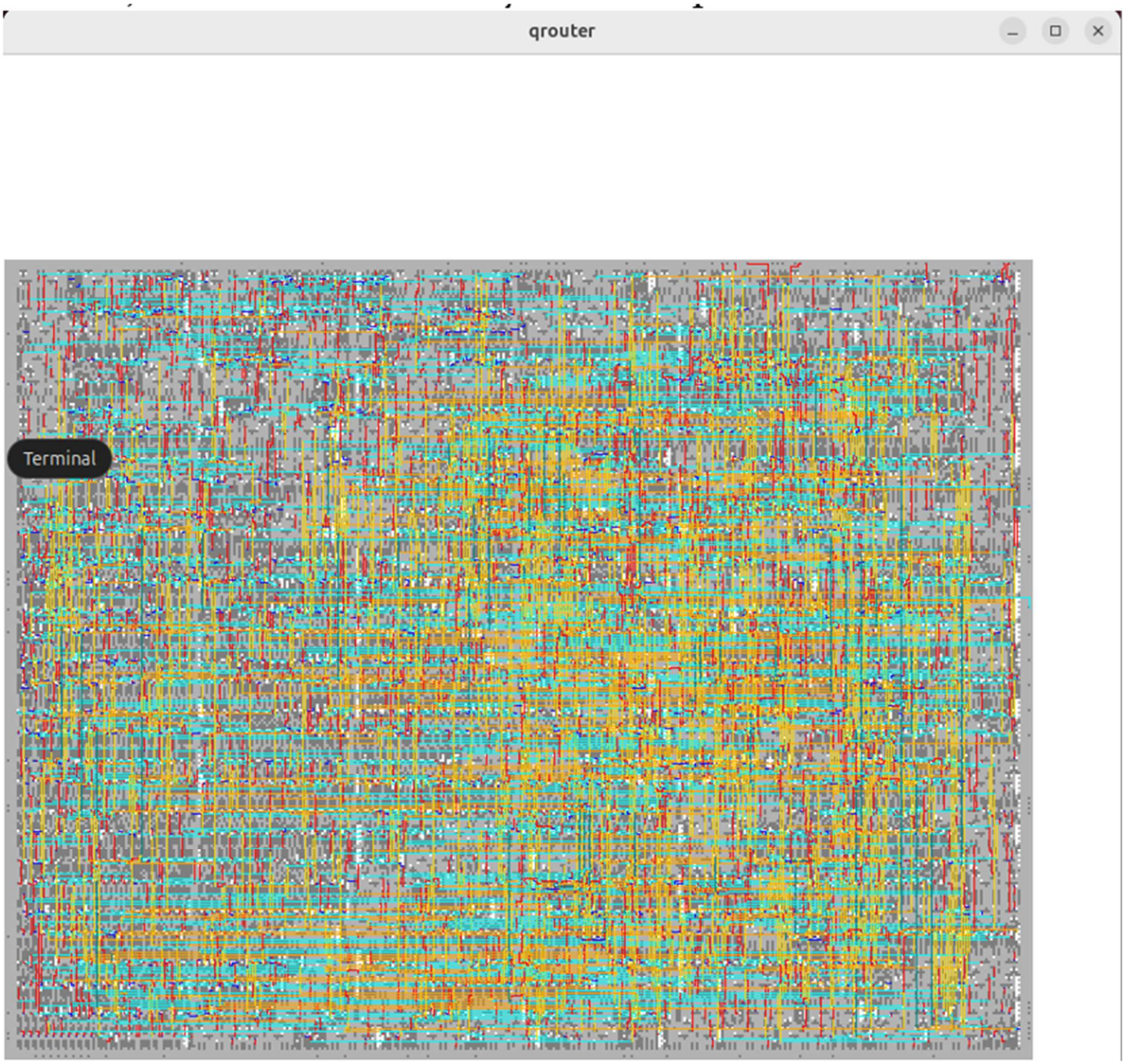

Qflow Manager
User: bhargav
Project: bhargav
/home/bhargav/bhargav
Checklist
Preparation Okay Run Settings
Synthesis Okay Run Settings
Placement Okay Run Settings
Static Timing Analysis Okay Run Settings
Routing (not done) Run Settings
Post-Route STA (not done) Run Settings
Migration (not done) Run Settings
DRC (not done) Stop Settings
LVS (not done) Run Settings
GDS (not done) Run Settings
Cleanup (not done) Run Settings
Route Settings
Show graphic view:
Route layers: max
Stop flow after route success:
hold at destination = 9.64914
Quit
Edit Layout

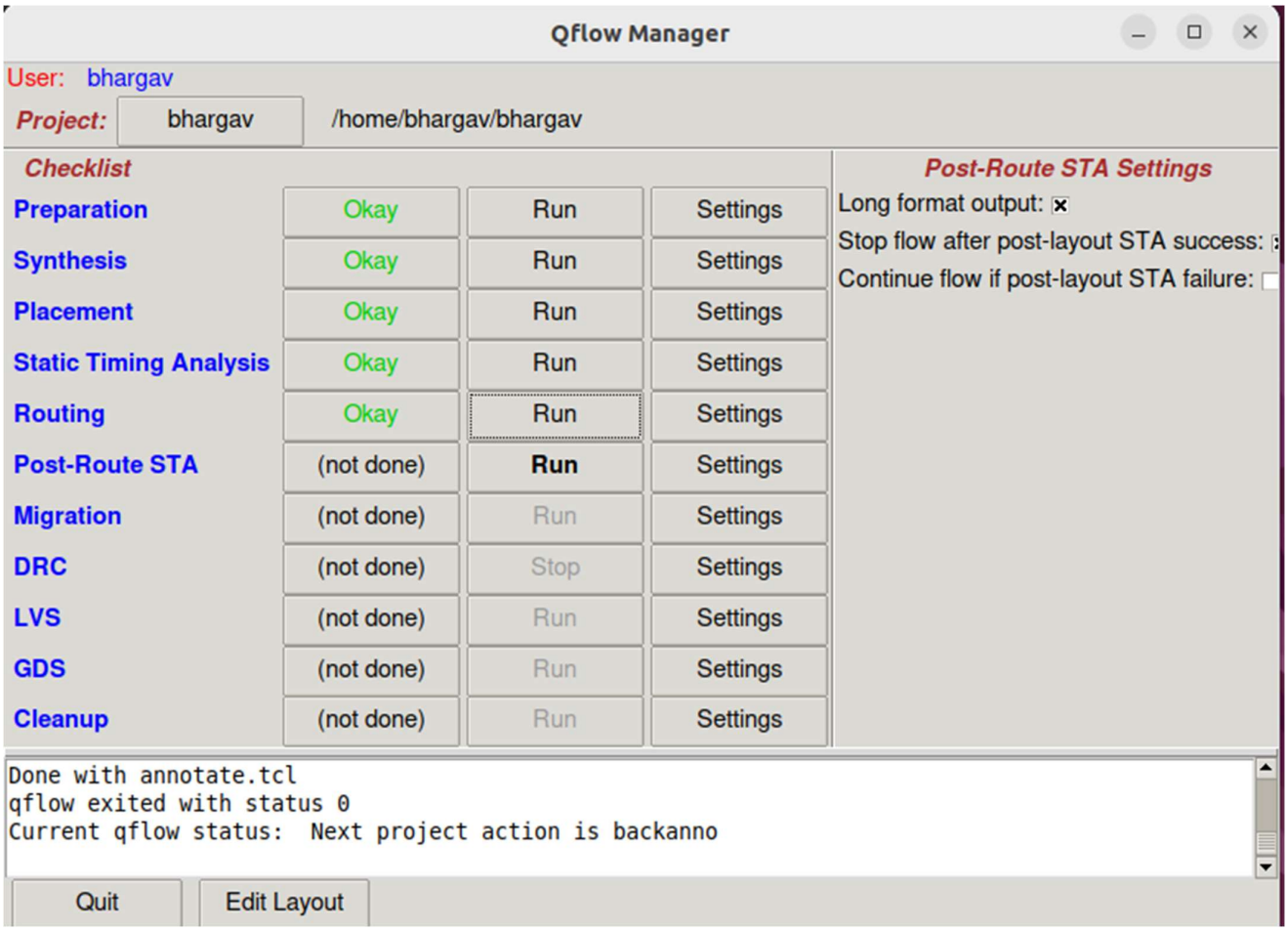
Routing

Routing is an important step in the VLSl physical design process where metal wires are used to connect the placed cells based on the circuit's netlist. After the placement stage decides the location of each cell, routing creates the actual paths for signals to travel between them. Routing is done in two phases: **global routing** and **detailed routing**. Global routing plans the general direction and space for each wire, while detailed routing assigns exact tracks and layers to complete the connections. The main goal is to connect all required nets correctly, while reducing wire length, avoiding congestion, and meeting timing requirements. In Qflow, Qrouter is the tool used for routing. It takes the placed design and connects the cells using a grid-based method across multiple metal layers. Routing must also follow specific design rules to avoid errors during chip manufacturing. Proper routing improves chip performance, reduces delays, and ensures the design works correctly in real hardware.

Qrouter

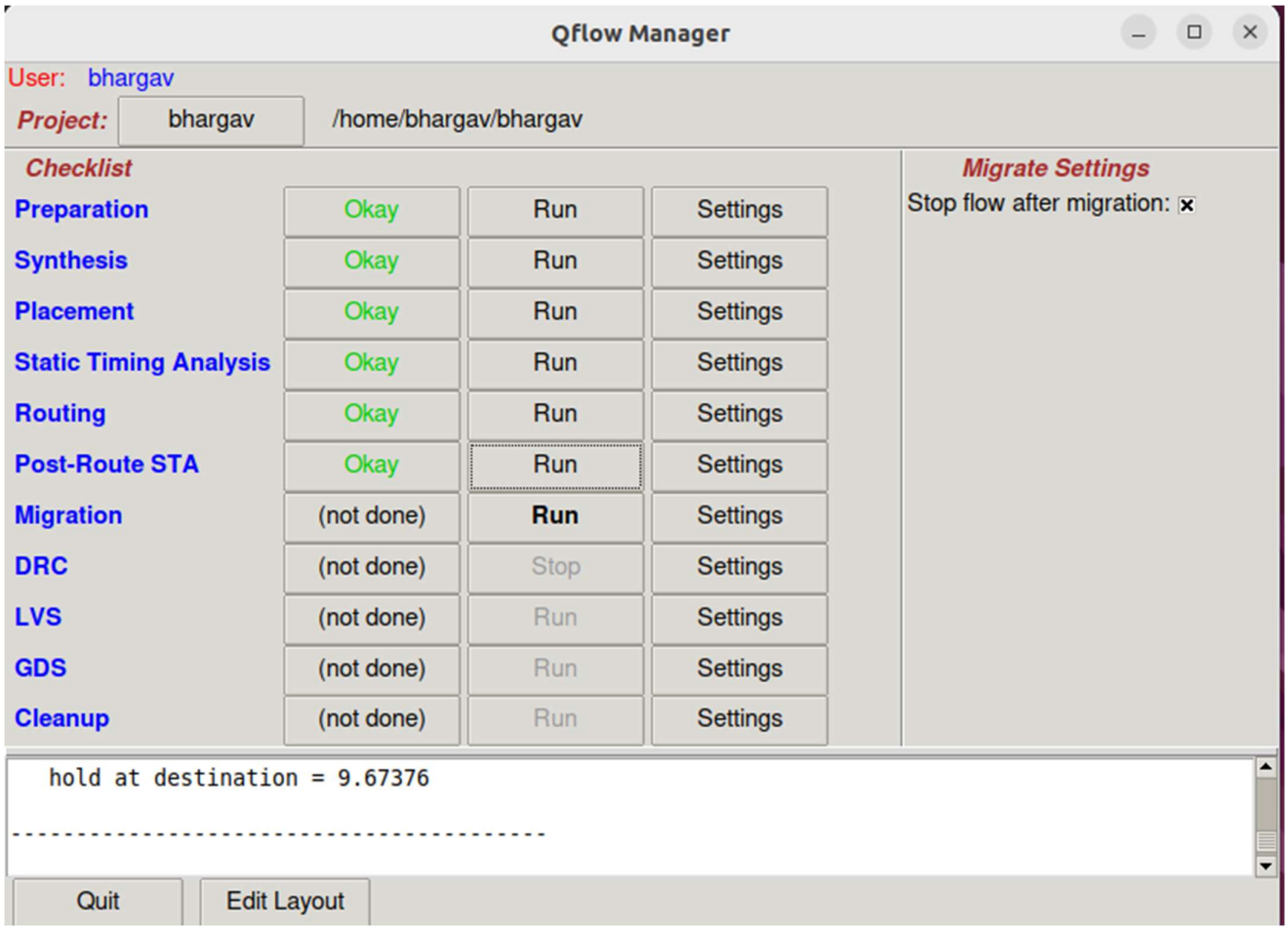
Qrouter is an open-source tool used in the Qflow digital design flow for the routing stage. It connects the logic cells placed on the chip by creating metal wire paths based on the netlist. Qrouter uses a grid-based routing method and supports multiple metal layers to make all required connections. The tool follows design rules to avoid errors like short circuits and ensures that the routing is clean and correct. It aims to reduce wire length, avoid congestion, and meet timing requirements. Qrouter plays a key role in completing the physical layout so that the chip can be fabricated successfully.

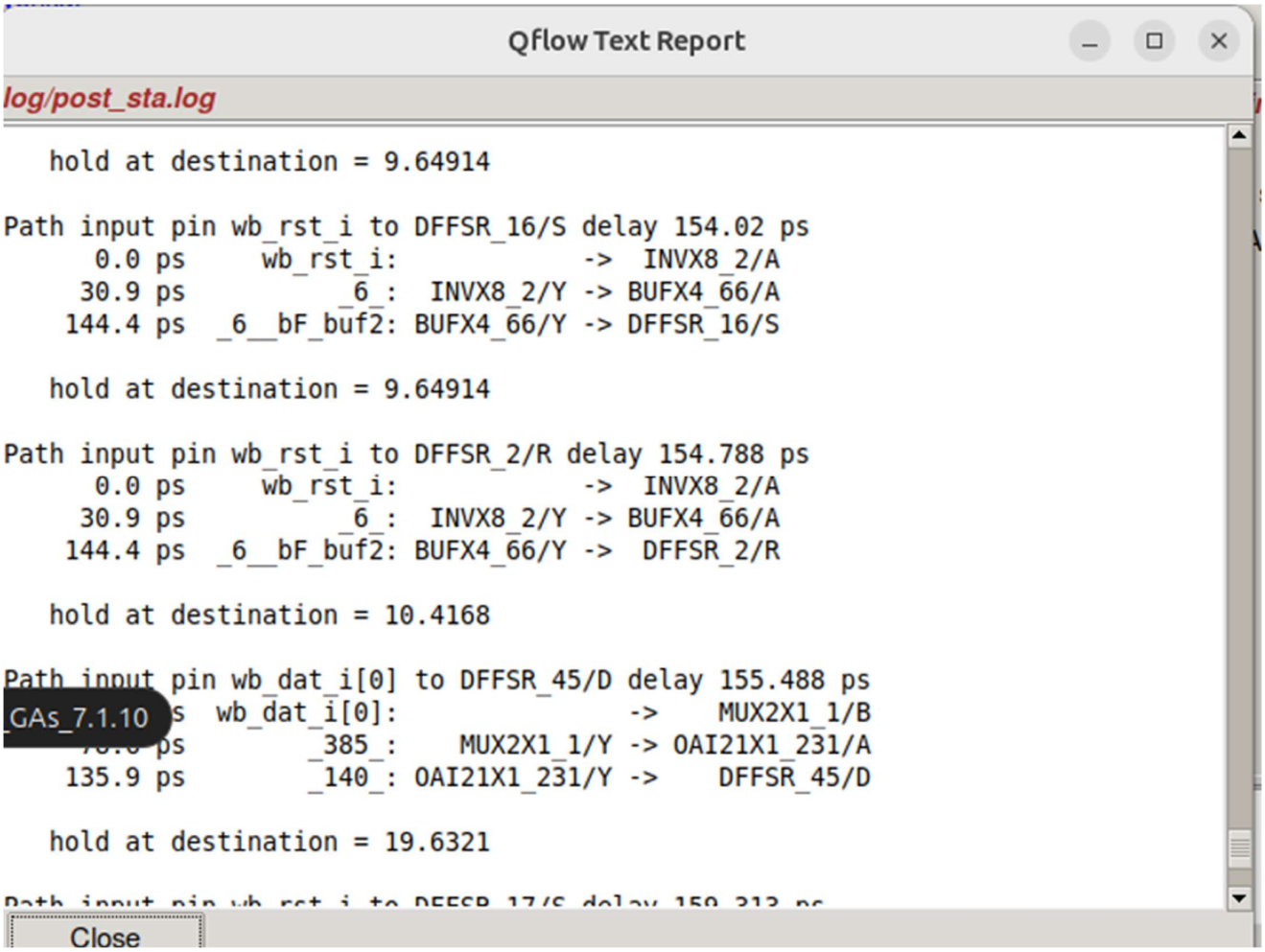

qrouter
Terminal



Post-Routing Static Timing Analysis (STA)

It is the process of verifying the timing performance of a digital circuit after the routing stage has been completed. At this stage, all physical information — including actual wire lengths, parasitics, and delays introduced by interconnects — is available, allowing for accurate timing calculations. The goal of post-routing STA is to ensure that the circuit meets timing requirements under all operating conditions without the need for dynamic simulation. It identifies critical paths and potential violations such as setup or hold time failures, which can affect the functional correctness of the design. In the Qflow design flow, tools like **vesta** are used to perform STA, analyzing the design based on a standard delay format (SDF) file and checking against the timing constraints specified by the user. This step is essential for timing closure, ensuring that the chip will operate reliably at the intended clock frequency.


Qflow Manager
User: bhargav
Project:
bhargav
/home/bhargav/bhargav
Checklist
Migrate Settings
Stop flow after migration:
Preparation
Okay
Run
Settings
Synthesis
Okay
Run
Settings
Placement
Okay
Run
Settings
Static Timing Analysis
Okay
Run
Settings
Routing
Okay
Run
Settings
Post-Route STA
Okay
Run
Settings
Migration
(not done)
Run
Settings
DRC
(not done)
Stop
Settings
LVS
(not done)
Run
Settings
GDS
(not done)
Run
Settings
Cleanup
(not done)
Run
Settings
hold at destination = 9.67376
Quit
Edit Layout


Qflow Text Report
log/post_sta.log
hold at destination = 9.64914
Path input pin wb_rst_i to DFFSR_16/S delay 154.02 ps
0.0 ps wb_rst_i: -> INVX8_2/A
30.9 ps _6_: INVX8_2/Y -> BUFX4_66/A
144.4 ps _6__bF_buf2: BUFX4_66/Y -> DFFSR_16/S
hold at destination = 9.64914
Path input pin wb_rst_i to DFFSR_2/R delay 154.788 ps
0.0 ps wb_rst_i: -> INVX8_2/A
30.9 ps _6_: INVX8_2/Y -> BUFX4_66/A
144.4 ps _6__bF_buf2: BUFX4_66/Y -> DFFSR_2/R
hold at destination = 10.4168
Path input pin wb_dat_i[0] to DFFSR_45/D delay 155.488 ps
wb_dat_i[0]: -> MUX2X1_1/B
_385_: MUX2X1_1/Y -> OAI21X1_231/A
135.9 ps _140_: OAI21X1_231/Y -> DFFSR_45/D
hold at destination = 19.6321
Close

Design Rule Checking (DRC)

Design Rule Checking (DRC) is an important step in the Qflow FPGA design process. It helps make sure that the physical layout of the chip follows all the rules set by the manufacturing process. These rules are important for ensuring that the chip can be built correctly and will work properly. DRC checks things like the distance between different parts of the layout, the width of wires, the size and placement of vias, and other important geometric rules. These checks help avoid problems that might cause the chip to fail during manufacturing or operation. Qflow often uses a tool called Magic to perform DRC. Magic is a well-known open-source software that helps detect errors in the layout. It finds mistakes that might cause functional failures or manufacturing defects.

By using DRC early in the design flow, designers can catch and fix issues before they become costly problems. This helps reduce the risk of needing rework or extra design changes later on. Effective DRC is essential for a successful and reliable chip design. Solving the error in drc during qflow processes.

Step 1: Click on the Edit Layout option.

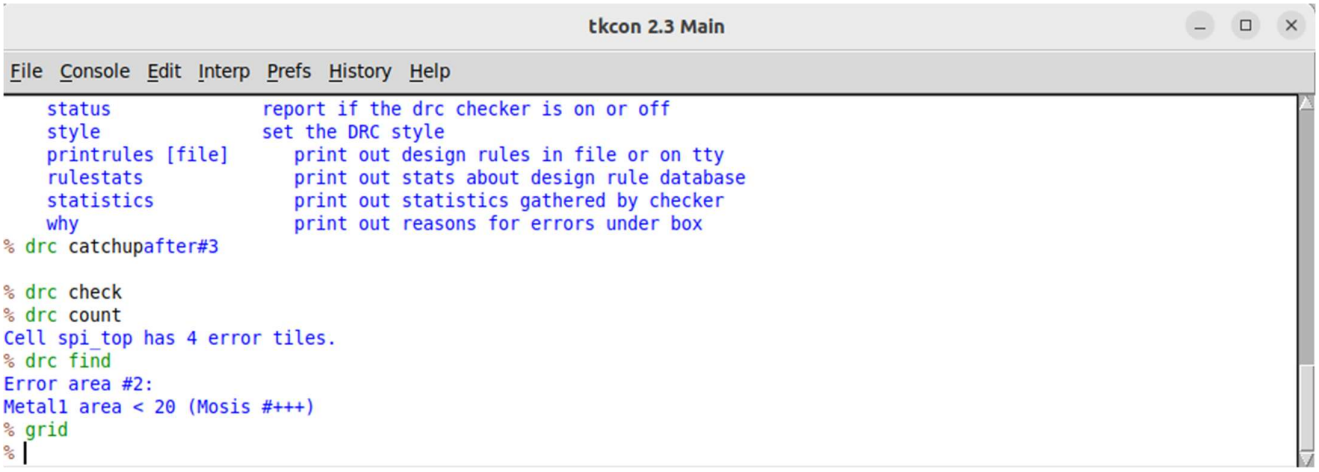
Step 2: In the tkcon window first type the command drc help in the main console window.

Step 3: Then type the following command :drc catchup, drc check, drc count, drc find in the order.

Step 4: type grid. Now a grid will open in layout. press Ctrl+z in the layout.

Step 5: fix the error.

Step 6: save and run the drc again.



What is Magic?

Magic is a layout tool developed at UC Berkeley, and it is used to draw and edit layouts at the transistor level or standard cell level. It provides powerful features like:

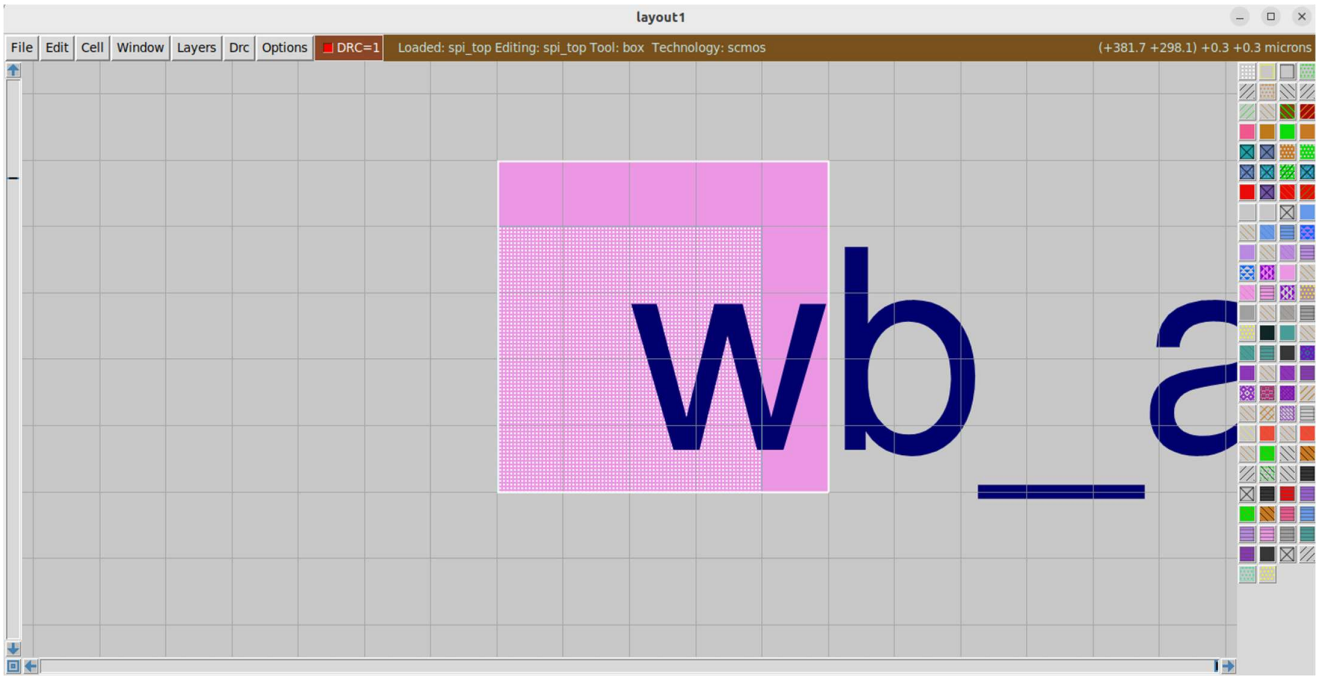
- Layout editing
- DRC (Design Rule Check)
- Extraction
- Simulation support
- LVS (Layout vs. Schematic)

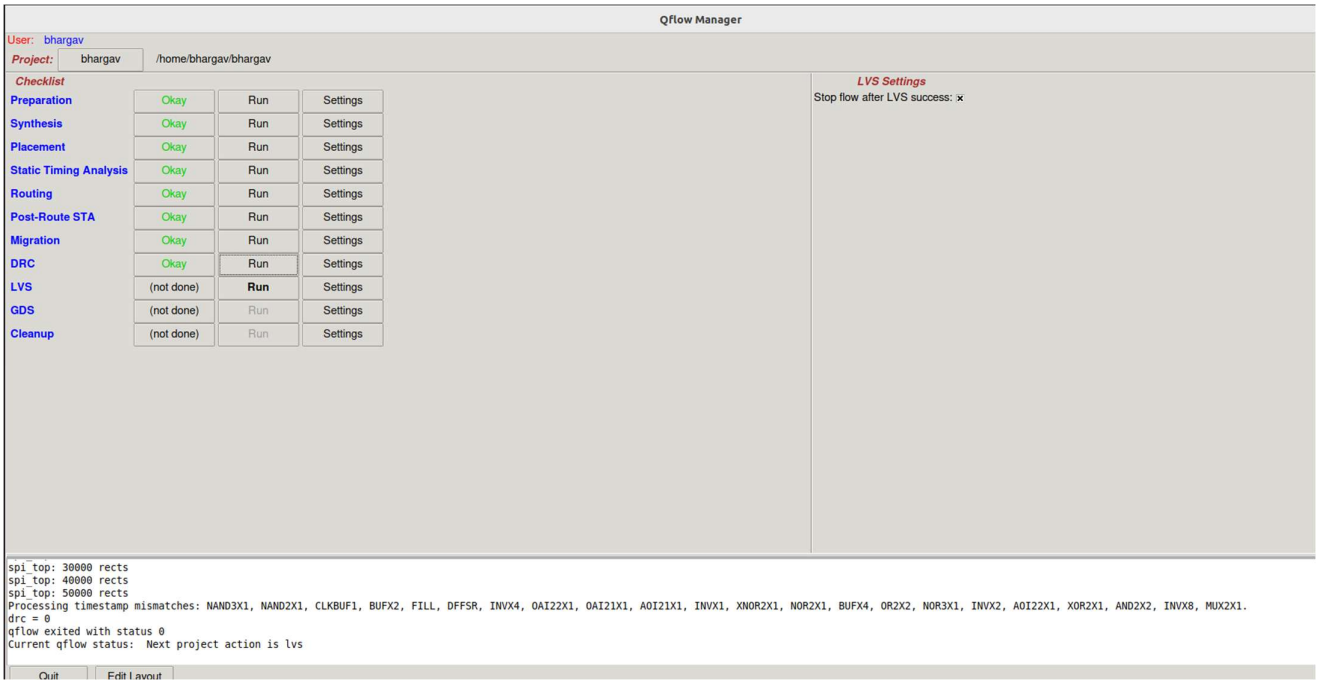
DRC Features in Magic

1. **Interactive DRC Checking**
   As you draw or modify the layout, Magic immediately checks for design rule violations. Errors are shown directly in the layout window (usually in red boxes).
2. **Live Feedback**
   You can use the tkcon console inside Magic to run DRC commands:
   - drc check – Checks for any rule violations.

- drc find – Highlights the location of errors.
- drc why – Gives details about the selected DRC error.
- drc count – Shows the total number of DRC violations.

3. **Error Highlighting**
   Magic visually marks error regions with a red outline or bounding box, making it easy to fix them.
4. **Grid Support for Area Checks**
   The grid command helps you visualize layout sizes. It's especially helpful when fixing errors .




Qflow Manager
User: bhargav
Project: bhargav
/home/bhargav/bhargav
Checklist
Preparation Okay Run Settings
Synthesis Okay Run Settings
Placement Okay Run Settings
Static Timing Analysis Okay Run Settings
Routing Okay Run Settings
Post-Route STA Okay Run Settings
Migration Okay Run Settings
DRC Okay Run Settings
LVS (not done) Run Settings
GDS (not done) Run Settings
Cleanup (not done) Run Settings
LVS Settings
Stop flow after LVS success:
spi_top: 30000 rects
spi_top: 40000 rects
spi_top: 50000 rects
Processing timestamp mismatches: NAND3X1, NAND2X1, CLKBUF1, BUFX2, FILL, DFFSR, INVX4, OAI22X1, OAI21X1, AOI21X1, INVX1, XNOR2X1, NOR2X1, BUFX4, OR2X2, NOR3X1, INVX2, AOI22X1, XOR2X1, AND2X2, INVX8, MUX2X1.
drc = 0
qflow exited with status 0
Current qflow status: Next project action is lvs
Quit
Edit Layout

LVS (Layout Vs Schematic)

Layout Versus Schematic (LVS) is an essential verification step in the physical design process. It ensures that the physical layout of a circuit exactly matches its original schematic design. LVS compares the netlist extracted from the layout — which shows how the components are actually connected on the chip — with the schematic netlist, which shows how they were intended to be connected logically. This process helps to find any differences between the layout and the schematic, such as missing or wrongly connected components. These errors, if left undetected, can lead to functional failures or manufacturing problems. To perform LVS, special verification tools are used. These tools check the connectivity by comparing the physical layout with the logical schematic. By doing this, it becomes easier to catch errors early in the design process and fix them before fabrication. This helps to reduce the chances of design rework and improves the final product's reliability. In the Qflow design flow for FPGA, Layout Versus Schematic (LVS) checking is done using a tool called Magic.

Magic is a free, open-source software used for VLSI layout, and it includes features for performing LVS checks. This step is important to make sure that the physical layout of the circuit matches the original design written in the schematic. Magic works by creating a netlist from the layout and comparing it with the schematic netlist. This helps find any mistakes, like missing wires, wrong connections, or parts that are not connected properly. Such errors can cause problems in how the chip works or even lead to manufacturing issues.Using Magic in Qflow helps designers confirm that the physical layout is correct and matches the intended design. This reduces the chance of errors and helps make sure the final chip works well and meets all design goals.

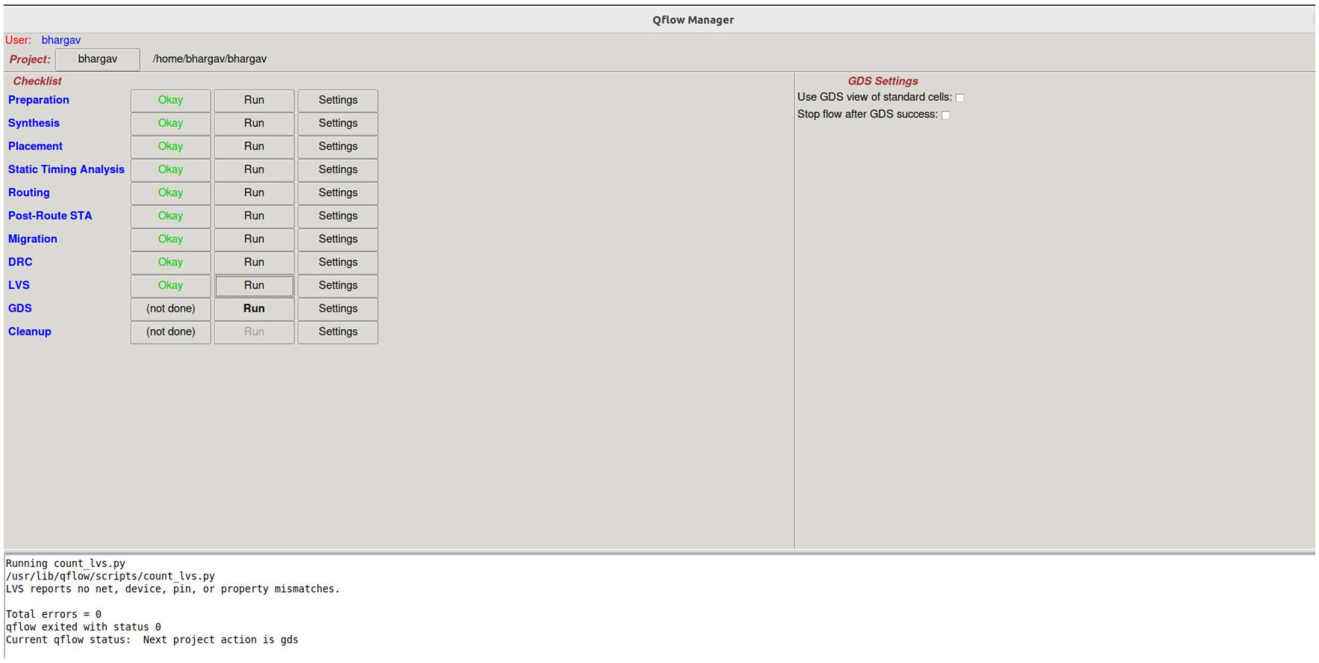
LVS Tool – Netgen

In open-source VLSI workflows like **Qflow** or **OpenLane**, **Netgen** is the tool used to perform LVS checks.

Features of Netgen:

- Compares a **SPICE or Verilog netlist** (from synthesis) with a **SPICE netlist** extracted from the layout.
- Reports **matches and mismatches** in terms of devices, connections, and pins.
- Easy to integrate with Magic for layout extraction.

Lightweight and widely used in academia and open-source VLSI projects.

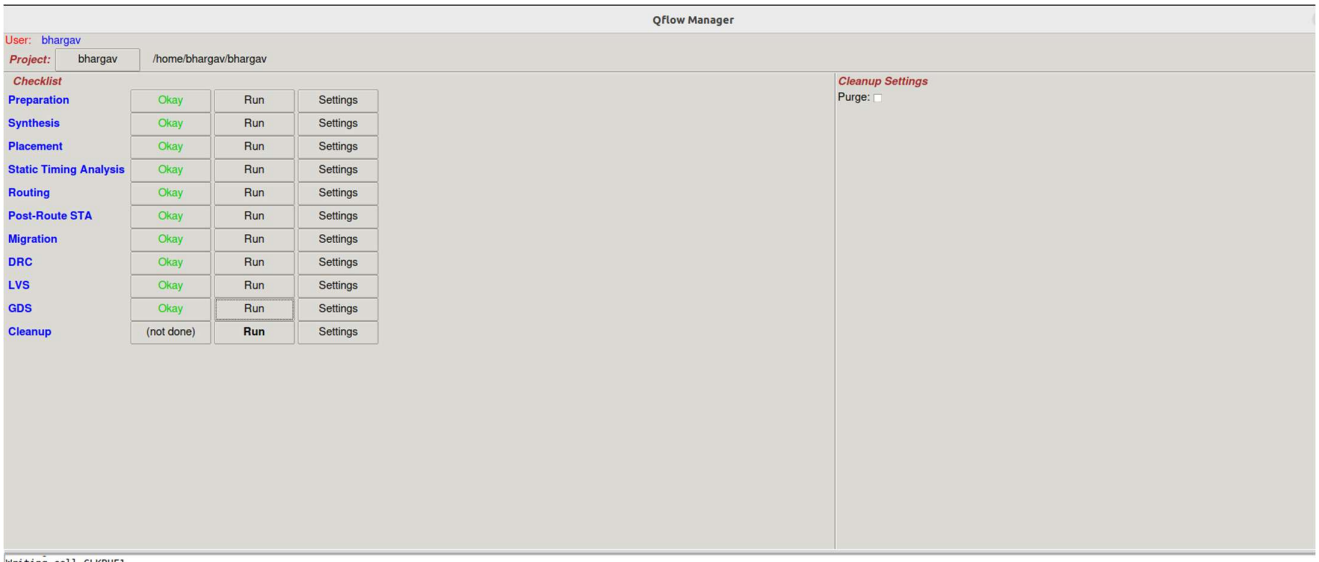


GDS (Graphic Data System)

In the VLSI design process, GDS (Graphic Data System), also known as GDSII, is a standard file format used to represent the final **physical layout of an integrated circuit (IC)**. After completing all stages such as synthesis, placement, and routing, the complete design is saved in a GDS file. This file contains all the physical information of the chip, such as the shapes and positions of transistors, wires, vias, and other layout elements spread across multiple layers of the chip.The GDS file acts **like a blueprint for manufacturing**. It ensures that the chip layout is communicated correctly to the fabrication plant. It captures both the geometry and connectivity of the circuit. This allows the manufacturing process to interpret and reproduce the exact chip design on silicon.

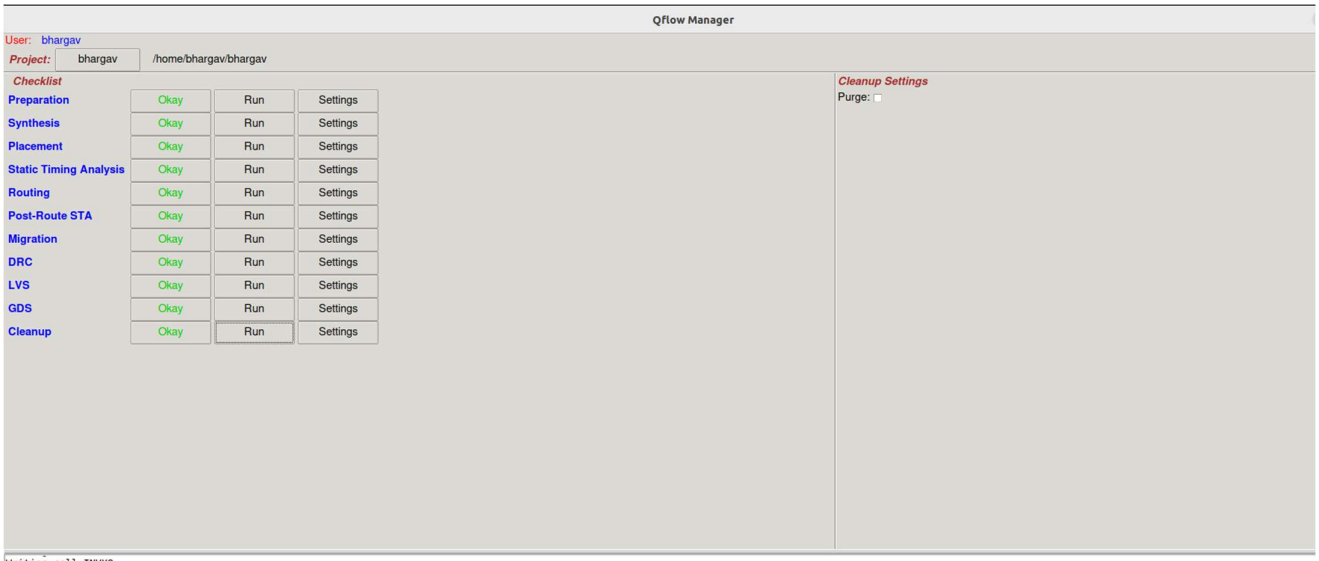
GDS files usually come with the extension .gds or .gdsii. They are very important for creating photomasks, which are used during photolithography, one of the key steps in semiconductor fabrication. These photomasks are like stencils that help print the chip design onto silicon wafers. In the Qflow FPGA design toolchain, tools like Magic are used to generate GDS files. Magic takes the final layout data and converts it into the GDS format. This step ensures that the physical layout matches the schematic design and is ready for fabrication. Creating and using GDS files is a crucial step in the physical design flow because it connects the digital design process with the actual manufacturing process. It gives a clear and accurate representation of the chip that can be used in further stages without confusion or errors.

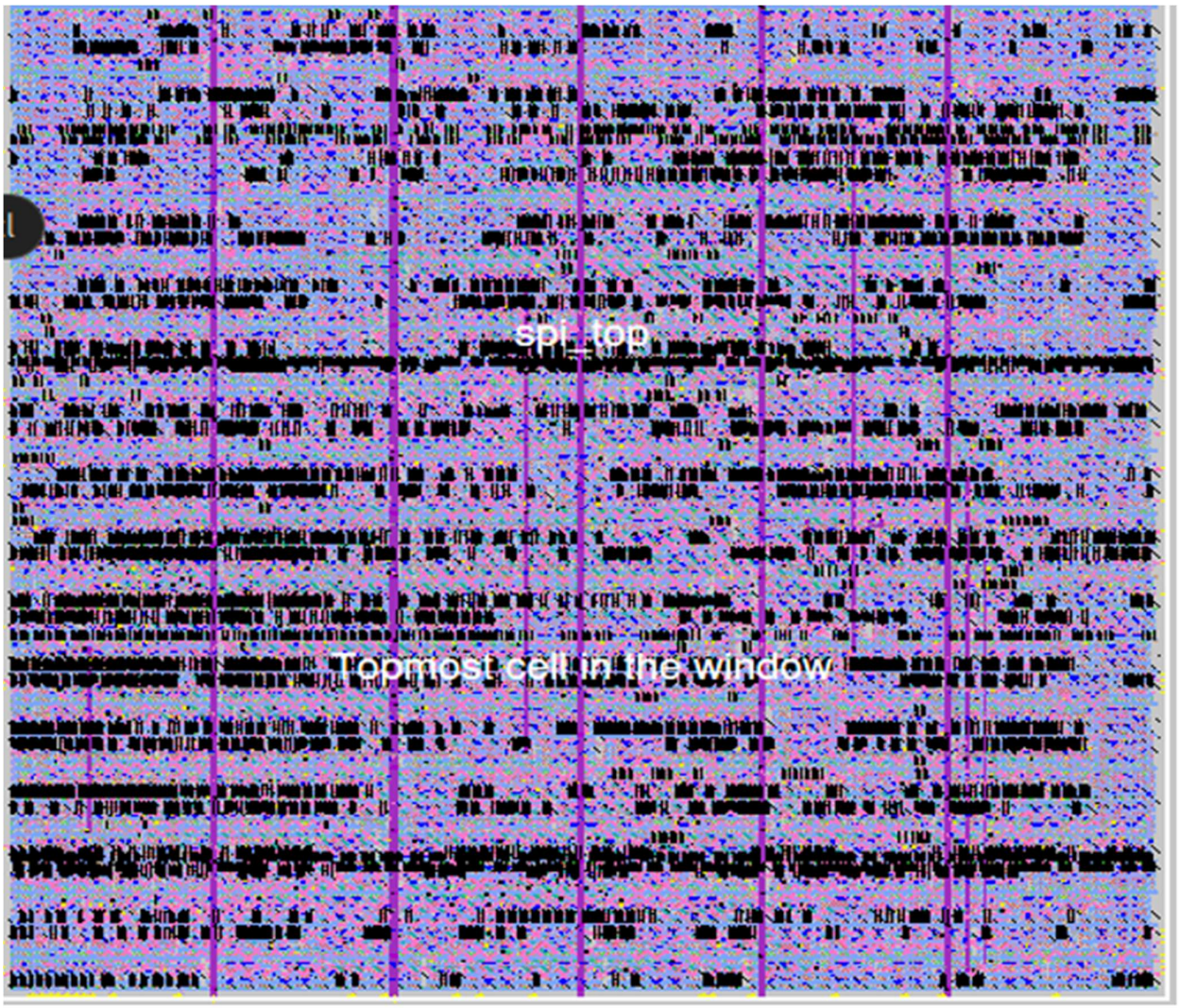
In summary, the GDS file ensures that the final chip will function as designed. It reduces the risk of mistakes during fabrication and ensures high reliability and performance in the finished semiconductor device.




```
Writing cell CLKBUF1
Writing cell MUX2X1
Writing cell INVX8
Writing cell BUFX4
Writing cell AOI22X1
qflow exited with status 0
Current qflow status:  Next project action is clean
```

Final Output View





Finally, the Qflow generates the **GDS file** with(.gds extension) which is a standard format used to represent the physical layout of an integrated circuit this file can be sent to the **semiconductor foundry** for fabrication.

The Art of Digital Synthesis using Qflow

– A Report by

Y.BHARGAV